

KDRSolvers:

Scalable, Flexible,
Task-Oriented Krylov Solvers

PAW-ATM 2025
David K. Zhang
Stanford University

$Ax = b$ Done Right

Software Lunch 2024-02-08

David K. Zhang

Stanford University

$$\mathbf{Ax} = \mathbf{b}$$

$$\mathbf{Ax} = \mathbf{b}$$

system of linear equations

known matrix



$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

system of linear equations

known matrix



$$\mathbf{Ax} = \mathbf{b}$$

known vector



system of linear equations

known matrix

unknown vector

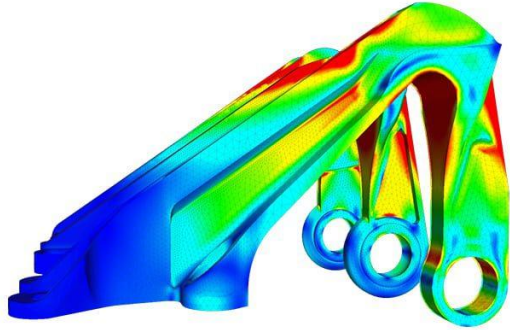
known vector

$$\mathbf{A}\mathbf{x} = \mathbf{b}$$

system of linear equations

$A\mathbf{x} = \mathbf{b}$ is everywhere

$A\mathbf{x} = \mathbf{b}$ is everywhere



$Ax = b$ is everywhere

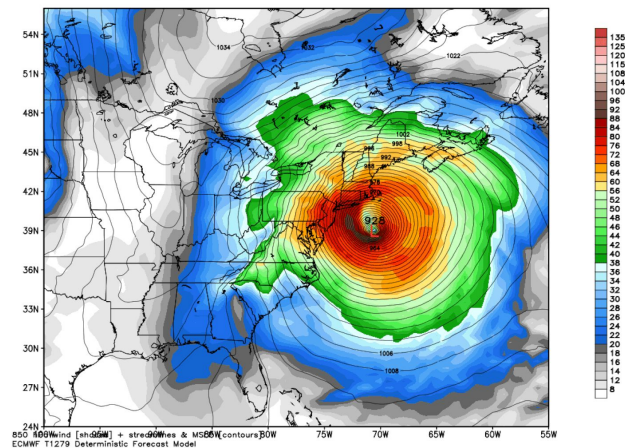


$Ax = b$ is everywhere



ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

Min/Max SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 97.0 knots

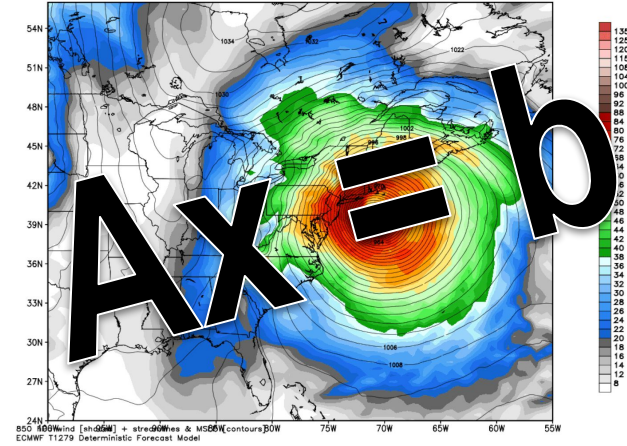


$Ax = b$ is everywhere



ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

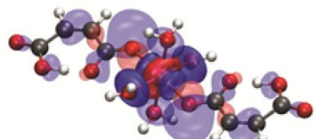
Min/Max SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 97.0 knots



Ax = b is everywhere

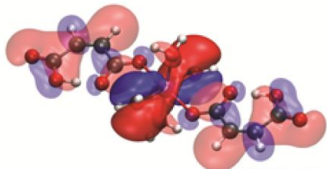


Antibonding



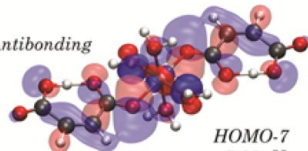
HOMO-3
-6.61 eV

Bonding



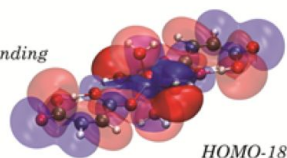
HOMO-24
-11.00 eV

Antibonding



HOMO-7
-7.95 eV

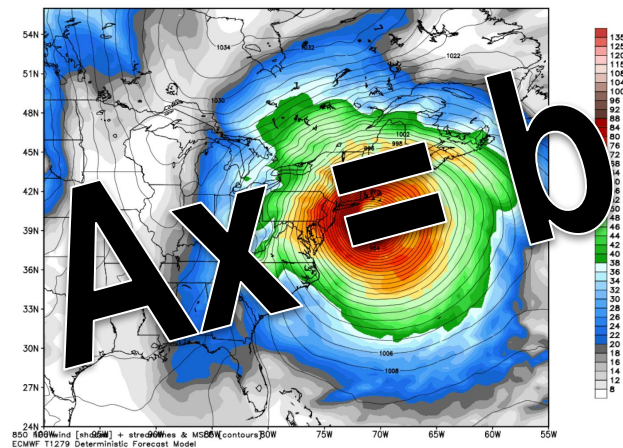
Bonding



HOMO-18
-10.44 eV

ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

Min/Max SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 87.0 knots

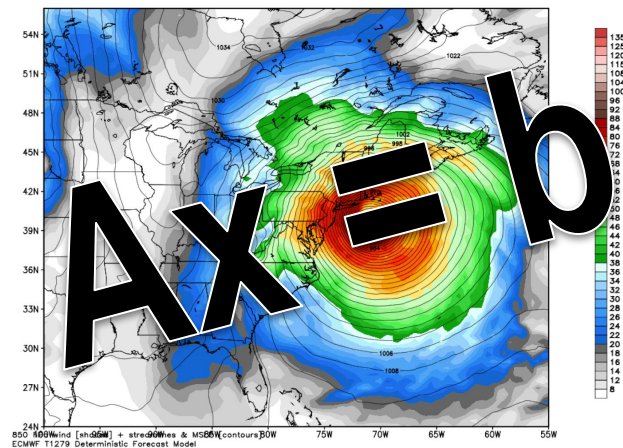


$Ax = b$ is everywhere

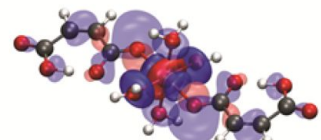


ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

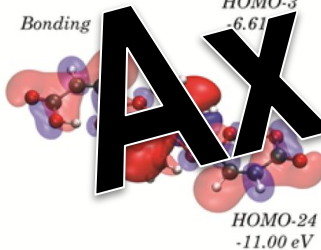
WinMax SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 87.0 knots



Antibonding



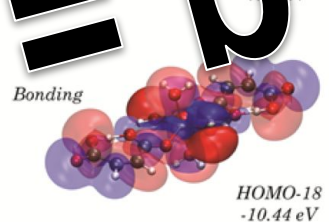
Bonding



Antibonding



Bonding

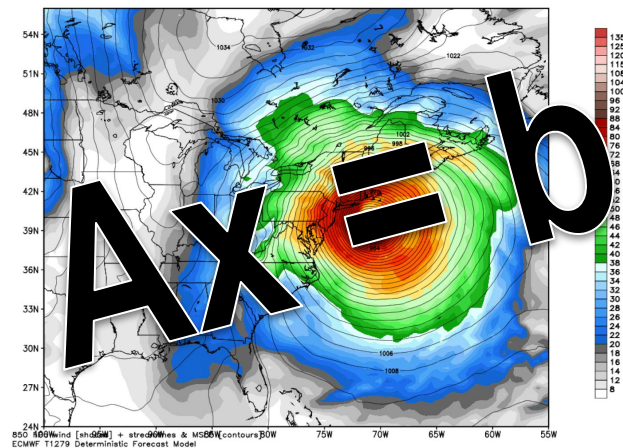


Ax = b is everywhere

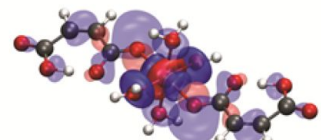


ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

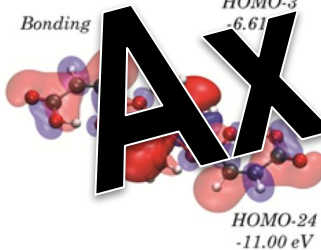
Min/Max SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 87.0 knots



Antibonding



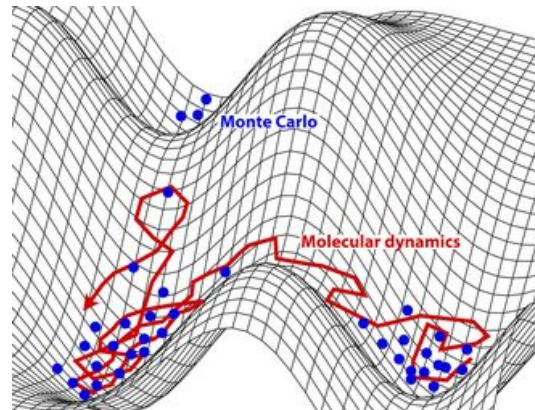
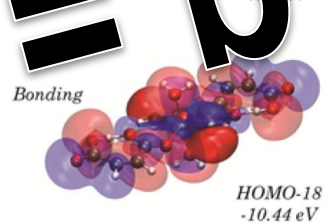
Bonding



Antibonding



Bonding

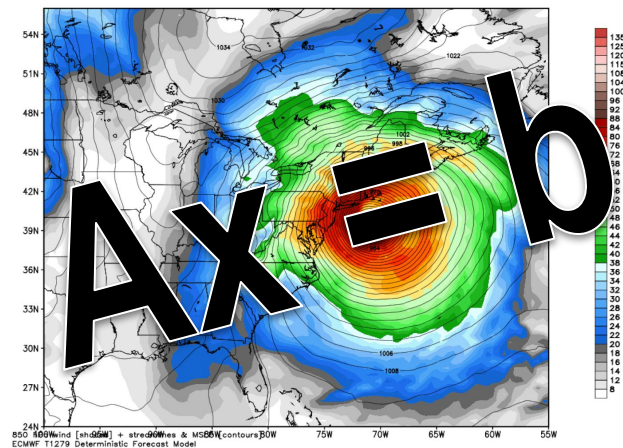


$Ax = b$ is everywhere

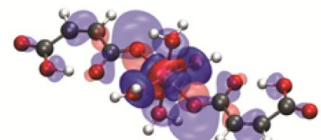


ECMWF 850 hPa Wind Speed [knots] and MSLP [hPa]
Init: 12Z22OCT2012 -- [216] hr --> Valid Wed 12Z31OCT2012

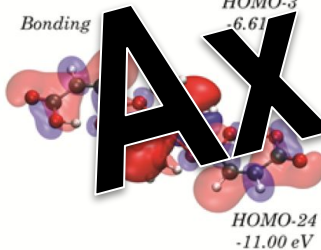
Min/Max SLP: 927.8 hPa | 1036.6 hPa
MaxWind: 87.0 knots



Antibonding



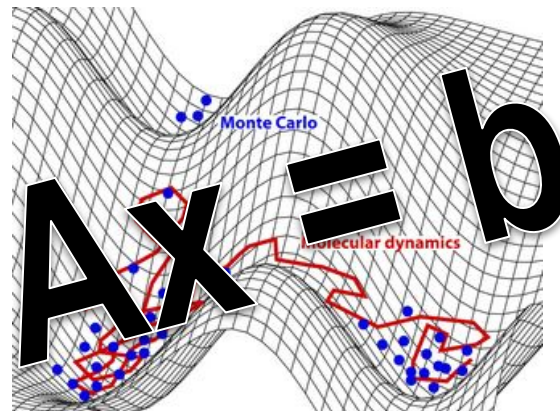
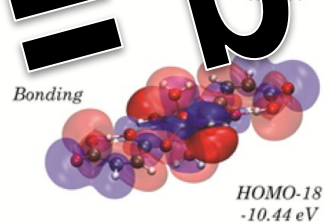
Bonding



Antibonding



Bonding



$A\mathbf{x} = \mathbf{b}$ is a matter of national pride

$A\mathbf{x} = \mathbf{b}$ is a matter of national pride



$A\mathbf{x} = \mathbf{b}$ is a matter of national pride



$Ax = b$ is a matter of national pride



Rank	TOP500 Rank	System	Cores	Rmax (PFlop/s)	HPCG (TFlop/s)
1	4	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	16004.50
2	1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	14054.00
3	5	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	4586.95
4	6	Leonardo - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy	1,824,768	238.70	3113.94
5	7	Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	2925.75

$A\mathbf{x} = \mathbf{b}$ has been done before

$A\mathbf{x} = \mathbf{b}$ has been done before



$A\mathbf{x} = \mathbf{b}$ has been done before



$A\mathbf{x} = \mathbf{b}$ has been done before



... but $A\mathbf{x} = \mathbf{b}$ can be done **better**

... but $A\mathbf{x} = \mathbf{b}$ can be done **better**

- **Parallel composition:**

What if I want to solve $A\mathbf{x} = \mathbf{b}$ in parallel with other work?

... but $A\mathbf{x} = \mathbf{b}$ can be done **better**

- **Parallel composition:**

What if I want to solve $A\mathbf{x} = \mathbf{b}$ in parallel with other work?

- **Format genericity:**

What if my matrix A is stored in a format you don't support?

... but $Ax = b$ can be done **better**

- **Parallel composition:**

What if I want to solve $Ax = b$ in parallel with other work?

- **Format genericity:**

What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**

What if I want to try out different data partitioning strategies?

... but $Ax = b$ can be done **better**

- **Parallel composition:**

What if I want to solve $Ax = b$ in parallel with other work?

- **Format genericity:**

What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**

What if I want to try out different data partitioning strategies?

- **Layout flexibility:**

What if my matrix A lives in more than one place?

... but $Ax = b$ can be done **better**

- **Parallel composition:** **task parallelism**

What if I want to solve $Ax = b$ in parallel with other work?

- **Format genericity:**

What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**

What if I want to try out different data partitioning strategies?

- **Layout flexibility:**

What if my matrix A lives in more than one place?

... but $Ax = b$ can be done **better**

- **Parallel composition:** **task parallelism (free, ty Legion)**

What if I want to solve $Ax = b$ in parallel with other work?

- **Format genericity:**

What if my matrix A is stored in a format you don't support?

- **Partitioning-awareness:**

What if I want to try out different data partitioning strategies?

- **Layout flexibility:**

What if my matrix A lives in more than one place?

... but $Ax = b$ can be done **better**

- **Parallel composition: task parallelism (free, ty Legion)**
What if I want to solve $Ax = b$ in parallel with other work?
- **Format genericity: K, D, R decomposition of storage formats**
What if my matrix A is stored in a format you don't support?
- **Partitioning-awareness:**
What if I want to try out different data partitioning strategies?
- **Layout flexibility:**
What if my matrix A lives in more than one place?

... but $Ax = b$ can be done **better**

- **Parallel composition: task parallelism (free, ty Legion)**
What if I want to solve $Ax = b$ in parallel with other work?
- **Format genericity: K, D, R decomposition of storage formats**
What if my matrix A is stored in a format you don't support?
- **Partitioning-awareness: **dependent partitioning****
What if I want to try out different data partitioning strategies?
- **Layout flexibility:**
What if my matrix A lives in more than one place?

... but $Ax = b$ can be done **better**

- **Parallel composition: task parallelism (free, ty Legion)**
What if I want to solve $Ax = b$ in parallel with other work?
- **Format genericity: K, D, R decomposition of storage formats**
What if my matrix A is stored in a format you don't support?
- **Partitioning-awareness: dependent partitioning**
What if I want to try out different data partitioning strategies?
- **Layout flexibility: multi-operator systems**
What if my matrix A lives in more than one place?

Sparse Matrix Storage Formats

Sparse Matrix Storage Formats

coo $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

<i>entry</i>	<i>row</i>	<i>col</i>
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

<i>entry</i>	<i>row</i>	<i>col</i>
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

<i>entry</i>	<i>col</i>	<i>rowptr</i>
3	0	[0, 1]
2	2	[2, 2]
4	1	[3, 4]
5	1	
1	2	

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

<i>entry</i>	<i>row</i>	<i>col</i>
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

<i>entry</i>	<i>col</i>
3	0
2	2
4	1
5	1
1	2

<i>rowptr</i>
[0, 1]
[2, 2]
[3, 4]

ELL $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$

<i>entry</i>	
3	2
4	0
5	1

<i>col</i>	
1	3
2	0
2	3

Sparse Matrix Storage Formats

COO

3

0

2

0

4

0

0

CSR

3

0

2

0

4

0

ELL

3

0

2

0

4

0

1

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

entry					[3, 4]		5 1		2 3	
3									3	
2									0	
4	1	1		4	1					
5	2	1		5	1					
1	2	2		1	2					

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$

ELL $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space K : nonzero entries of matrix

entry					[3, 4]					
3									3	
2									0	
4	1	1		4	1		5	1	2	3
5	2	1		5	1					
1	2	2		1	2					

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ **Key Ins**

CSR	3	0	2
	0	4	0

ELL $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ \text{are defined by} & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space K : nonzero entries of matrix
- Domain index space D : indices of input vector (columns)

entry		
3	- Kernel i - Domain	
2		
4	1	1
5	2	1
1	2	2

4	1
5	1
1	2

[3, 4]

5	1
---	---

)		p/	
			3
			0
	2		3

Sparse Matrix Storage Formats

COO $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & & \end{bmatrix}$

CSR $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \end{bmatrix}$

ELL $\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ & & 1 \end{bmatrix}$

Key Insight: all sparse matrix storage formats are defined by binary relations between three fundamental index spaces:

- Kernel index space K : nonzero entries of matrix
- Domain index space D : indices of input vector (columns)
- Range index space R : indices of output vector (rows)

entry		
3		
2		
4	1	1
5	2	1
1	2	2

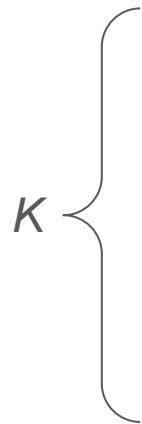
4	1
5	1
1	2

[3, 4]

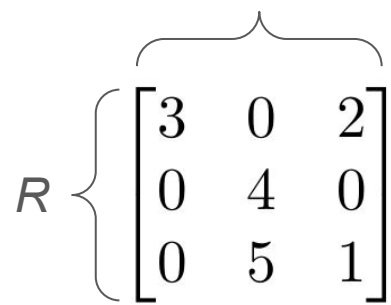
5	1
---	---

	3
	0
2	3

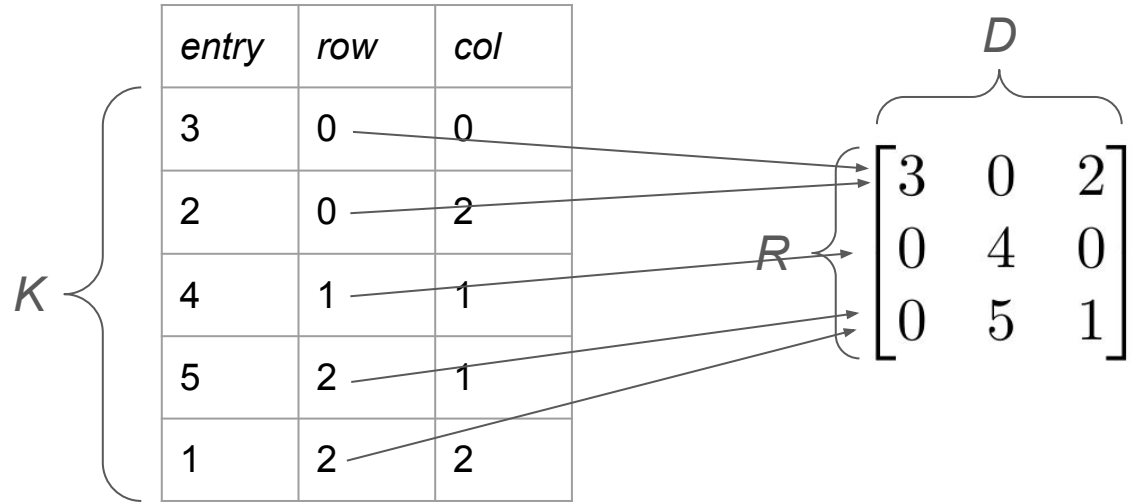
COO Sparse Matrix Format



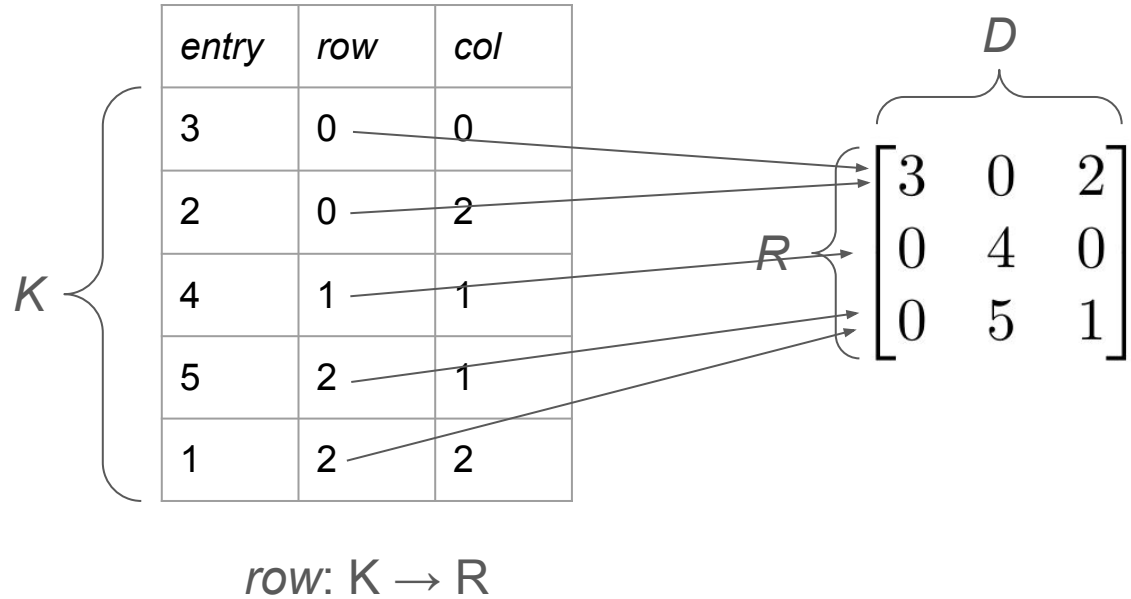
<i>entry</i>	<i>row</i>	<i>col</i>
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2


$$R \left\{ \begin{matrix} \overbrace{\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}}^D \end{matrix} \right.$$

COO Sparse Matrix Format



COO Sparse Matrix Format



COO Sparse Matrix Format

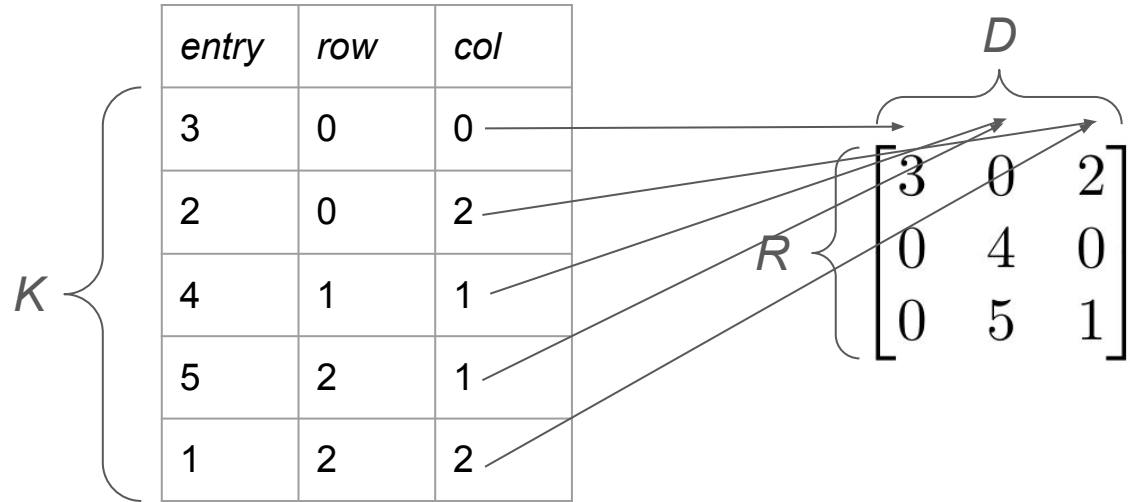
K {

<i>entry</i>	<i>row</i>	<i>col</i>
3	0	0
2	0	2
4	1	1
5	2	1
1	2	2

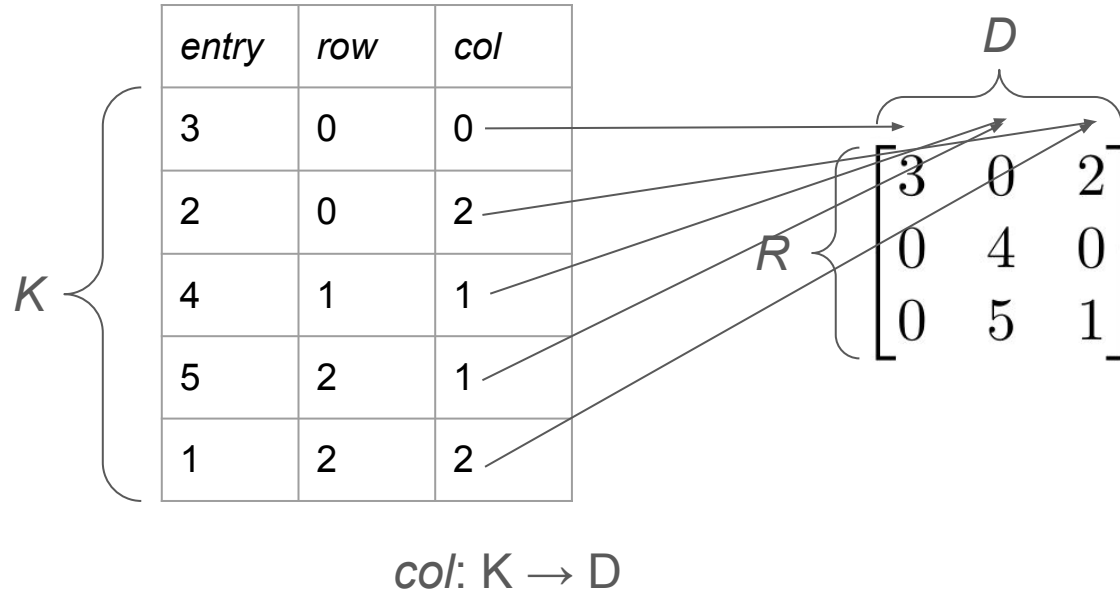
R { D {

$$\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$$

COO Sparse Matrix Format



COO Sparse Matrix Format



COO Sparse Matrix Format

	<i>entry</i>	<i>row</i>	<i>col</i>
K	3	0	0
	2	0	2
	4	1	1
	5	2	1
	1	2	2

$$R \left\{ \begin{matrix} \overbrace{\phantom{\begin{bmatrix} 3 & 0 & 2 \end{bmatrix}}}^D \\ \begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix} \end{matrix} \right.$$

COO format is defined by:

- **column relation** $col: K \rightarrow D$
- **row relation** $row: K \rightarrow R$

CSR Sparse Matrix Format

K {

<i>entry</i>	<i>col</i>
3	0
2	2
4	1
5	1
1	2

R {

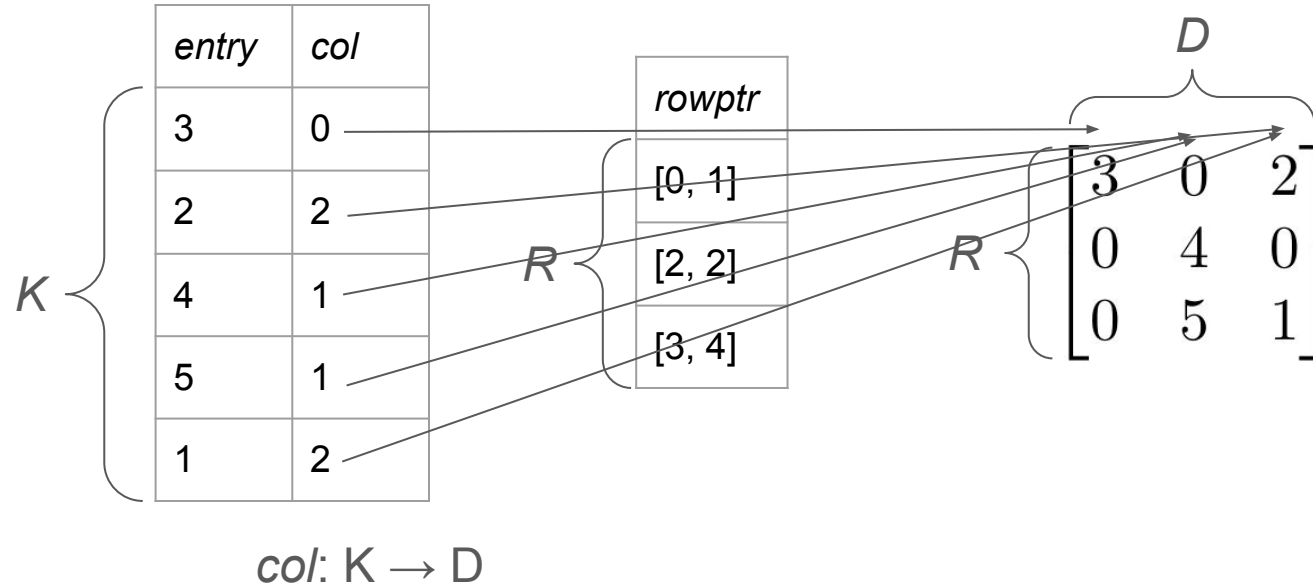
<i>rowptr</i>
[0, 1]
[2, 2]
[3, 4]

D

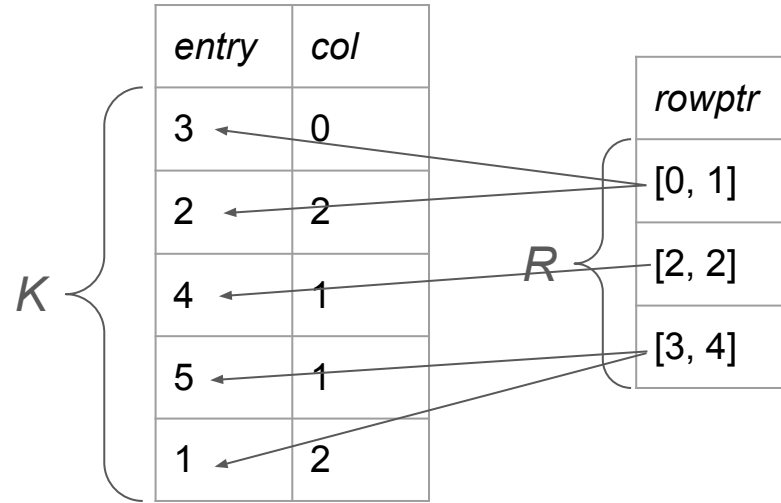
R {

$$\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$$

CSR Sparse Matrix Format

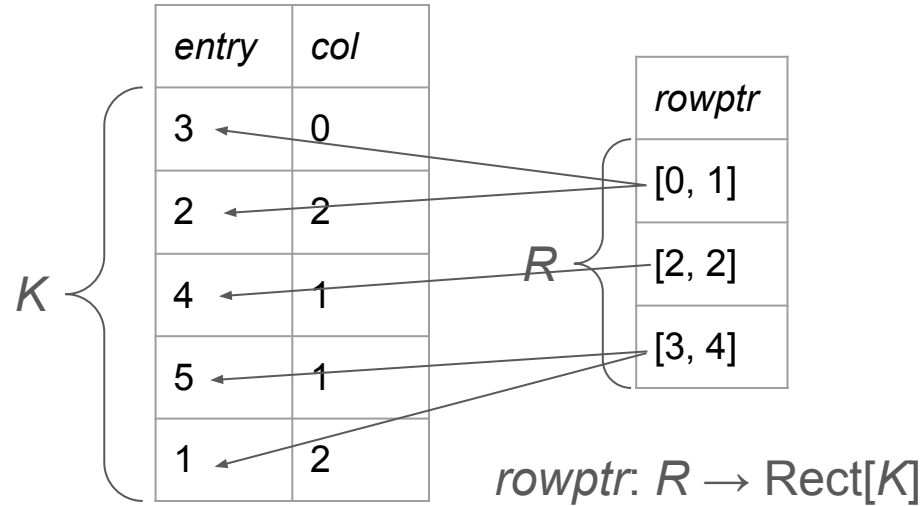


CSR Sparse Matrix Format



$$R \left\{ \begin{matrix} \overbrace{\left[\begin{matrix} 3 & 0 & 2 \end{matrix} \right]}^D \\ \left[\begin{matrix} 0 & 4 & 0 \\ 0 & 5 & 1 \end{matrix} \right] \end{matrix} \right.$$

CSR Sparse Matrix Format



$$R \left\{ \begin{matrix} D \\ \begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix} \end{matrix} \right.$$

CSR Sparse Matrix Format

K {

<i>entry</i>	<i>col</i>
3	0
2	2
4	1
5	1
1	2

R {

<i>rowptr</i>
[0, 1]
[2, 2]
[3, 4]

D

R {

$$\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}$$

CSR format is defined by:

- **column relation** *col*: $K \rightarrow D$
- **row relation** *rowptr*: $R \rightarrow \text{Rect}[K]$

ELL Sparse Matrix Format

R {	<i>entry</i>		<i>col</i>	
	3	2	1	3
	4	0	2	0
	5	1	2	3

$$R \left\{ \begin{matrix} \overbrace{\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}}^D \end{matrix} \right.$$

ELL Sparse Matrix Format

$$K = R \times N$$

<i>entry</i>		<i>col</i>	
3	2	1	3
4	0	2	0
5	1	2	3

$$R \left\{ \begin{matrix} \overbrace{\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}}^D \end{matrix} \right.$$

ELL Sparse Matrix Format

$$K = R \times N$$

<i>entry</i>		<i>col</i>	
3	2	1	3
4	0	2	0
5	1	2	3

$$R \left\{ \begin{array}{c} \overbrace{\begin{bmatrix} 3 & 0 & 2 \\ 0 & 4 & 0 \\ 0 & 5 & 1 \end{bmatrix}}^D \end{array} \right.$$

ELL format is defined by:

- column relation $col: K \rightarrow D$
- **implicit** row relation $\pi_1: R \times N \rightarrow R$

K, D, R Storage Formats

Format	Structural Assumptions	Column Relation	Row Relation
Dense	$K = R \times D$	$\pi_2 : R \times D \rightarrow D$ (implicit)	$\pi_1 : R \times D \rightarrow R$ (implicit)
COO	(none)	$\text{col} : K \rightarrow D$	$\text{row} : K \rightarrow R$
CSR	K is totally ordered	$\text{col} : K \rightarrow D$	$\text{rowptr} : R \rightarrow [K, K]$
CSC	K is totally ordered	$\text{colptr} : D \rightarrow [K, K]$	$\text{row} : K \rightarrow R$
ELL	$K = R \times K_0$	$\text{col} : K \rightarrow D$	$\pi_1 : R \times K_0 \rightarrow R$ (implicit)
ELL'	$K = D \times K_0$	$\pi_1 : D \times K_0 \rightarrow D$ (implicit)	$\text{row} : K \rightarrow R$
DIA	$D = \{1, \dots, d\}$ $R = \{1, \dots, r\}$ $K = K_0 \times \{1, \dots, d\}$ $\text{offset} : K_0 \rightarrow \mathbb{Z}$	$\text{col} : (k_0, i) \mapsto i$ (implicit)	$\text{row} : (k_0, i) \mapsto i - \text{offset}(k_0)$ (implicit)
BCSR	$K = K_0 \times B_R \times B_D$ $D = D_0 \times B_D$ $R = R_0 \times B_R$ K_0 is totally ordered	$\text{col} : K_0 \rightarrow D_0$	$\text{rowptr} : R_0 \rightarrow [K_0, K_0]$
BCSC	$K = K_0 \times B_R \times B_D$ $D = D_0 \times B_D$ $R = R_0 \times B_R$ K_0 is totally ordered	$\text{colptr} : D_0 \rightarrow [K_0, K_0]$	$\text{row} : K_0 \rightarrow R_0$

Fig. 3. A wide variety of sparse matrix storage formats are realized in KDRSolvers as particular forms of column and row relations, subject to certain structural assumptions on the index spaces (K, D, R) . Some column and row relations, marked as (implicit), can be specified without additional metadata as a consequence of structural assumptions or other auxiliary data. Here, π_i denotes the canonical projection function from a Cartesian product onto its i th coordinate, and whenever an index space K is assumed to be totally ordered, we denote by $[K, K]$ the set of contiguous intervals in K .

K, D, R Storage Formats

To define a matrix storage format, you give me:

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
 - technically, we could derive this from the relations, but it would be slow

K, D, R Storage Formats

To define a matrix storage format, you give me:

- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
 - technically, we could derive this from the relations, but it would be slow

Then, LegionSolvers can automatically do **dependent partitioning**:

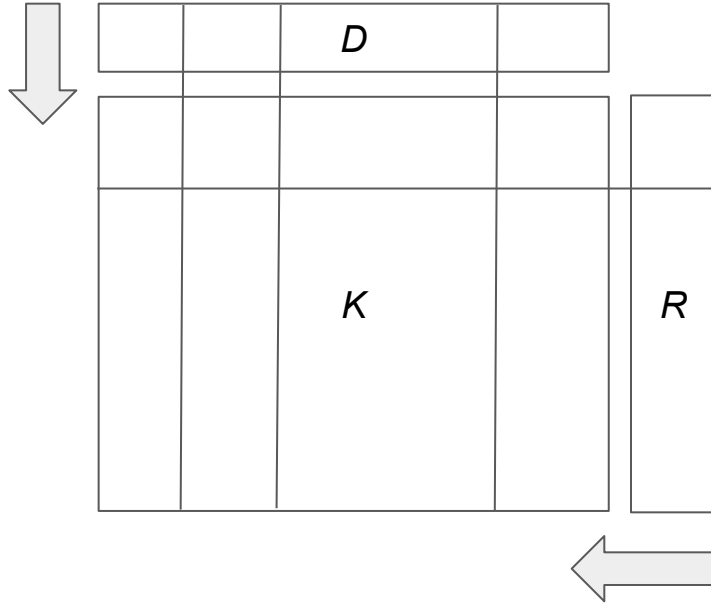
K, D, R Storage Formats

To define a matrix storage format, you give me:

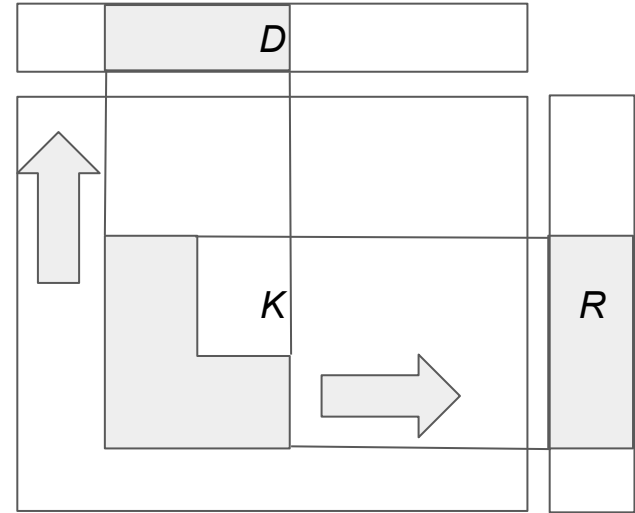
- **kernel region** containing nonzero entries
- **binary relations** $K \leftrightarrow D$ and $K \leftrightarrow R$
- **task** (function pointer) for matrix-vector multiplication
 - technically, we could derive this from the relations, but it would be slow

Then, LegionSolvers can automatically do **dependent partitioning**:
transferring a partition of one of (K, D, R) to the others by images/preimages

Matrix Dependent Partitioning Operators



Kernel partition from
domain and range partitions



Domain and range partitions
from kernel partition

Matrix Dependent Partitioning Operators

```
virtual Legion::IndexPartition
create_kernel_partition_from_domain_partition(
    Legion::IndexPartition domain_partition
) const = 0;

virtual Legion::IndexPartition create_kernel_partition_from_range_partition(
    Legion::IndexPartition range_partition
) const = 0;

virtual Legion::IndexPartition
create_domain_partition_from_kernel_partition(
    Legion::IndexSpace domain_space, Legion::IndexPartition kernel_partition
) const = 0;

virtual Legion::IndexPartition create_range_partition_from_kernel_partition(
    Legion::IndexSpace range_space, Legion::IndexPartition kernel_partition
) const = 0;
```

Multi-Operator Systems

Who says $A\mathbf{x} = \mathbf{b}$ only involves a single matrix A and vector $\mathbf{b}$?

Multi-Operator Systems

Who says $A\mathbf{x} = \mathbf{b}$ only involves a single matrix A and vector $\mathbf{b}$?

$$A_{11} \mathbf{x}_1 + A_{12} \mathbf{x}_2 + A_{13} \mathbf{x}_3 = \mathbf{b}_1$$

$$A_{21} \mathbf{x}_1 + A_{22} \mathbf{x}_2 + A_{23} \mathbf{x}_3 = \mathbf{b}_2$$

Multi-Operator Systems

Who says $\mathbf{Ax} = \mathbf{b}$ only involves a single matrix A and vector $\mathbf{b}$?

$$A_{11} \mathbf{x}_1 + A_{12} \mathbf{x}_2 + A_{13} \mathbf{x}_3 = \mathbf{b}_1$$

$$A_{21} \mathbf{x}_1 + A_{22} \mathbf{x}_2 + A_{23} \mathbf{x}_3 = \mathbf{b}_2$$

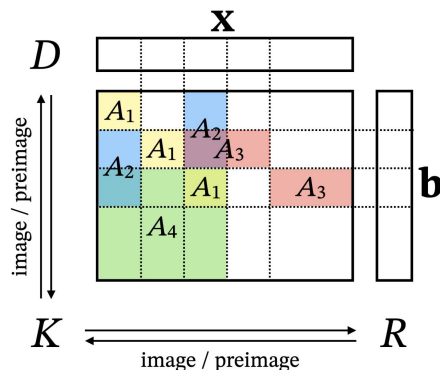
We allow a *logical* matrix or vector
to consist of multiple *physical* pieces

Multi-Operator Systems

Who says $A\mathbf{x} = \mathbf{b}$ only involves a single matrix A and vector $\mathbf{b}$?

$$\begin{aligned} A_{11} \mathbf{x}_1 + A_{12} \mathbf{x}_2 + A_{13} \mathbf{x}_3 &= \mathbf{b}_1 \\ A_{21} \mathbf{x}_1 + A_{22} \mathbf{x}_2 + A_{23} \mathbf{x}_3 &= \mathbf{b}_2 \end{aligned}$$

We allow a *logical* matrix or vector to consist of multiple *physical* pieces



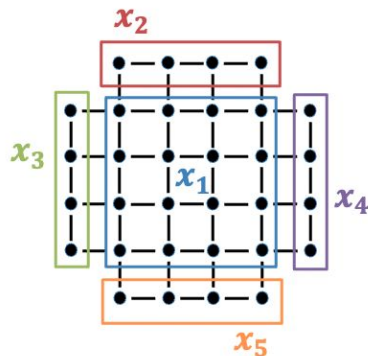
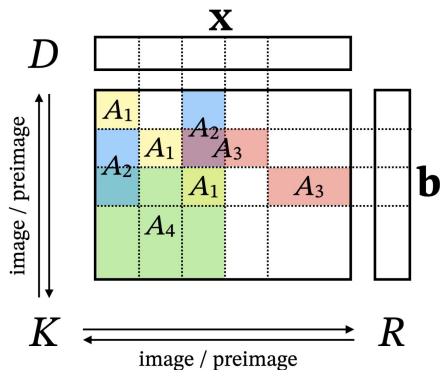
Multi-Operator Systems

Who says $A\mathbf{x} = \mathbf{b}$ only involves a single matrix A and vector $\mathbf{b}$?

$$\begin{aligned} A_{11} \mathbf{x}_1 + A_{12} \mathbf{x}_2 + A_{13} \mathbf{x}_3 &= \mathbf{b}_1 \\ A_{21} \mathbf{x}_1 + A_{22} \mathbf{x}_2 + A_{23} \mathbf{x}_3 &= \mathbf{b}_2 \end{aligned}$$

We allow a *logical* matrix or vector to consist of multiple *physical* pieces

Use data wherever it naturally exists in the context of your application



An example of **heterogeneous dimension**: a 2D grid with four 1D boundary pieces. Note that we can easily enforce periodic boundary conditions by identifying $x_2 = x_5$ and $x_3 = x_4$.

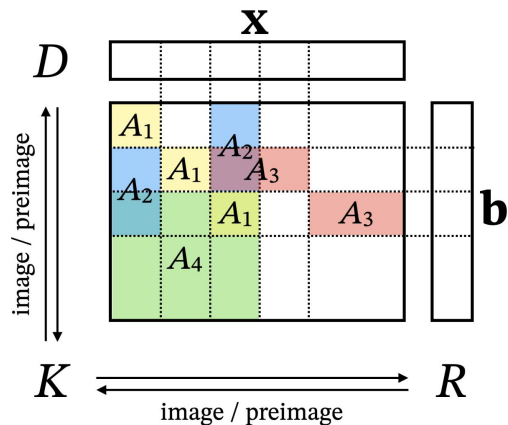
Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

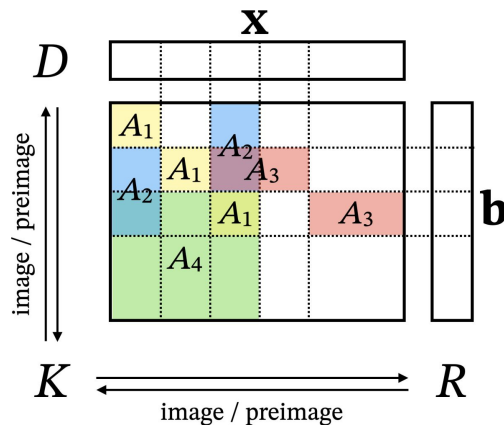
For matrix-vector products, we launch a separate task for each matrix piece and reduce into a single vector



Operations on Multi-Operator Systems

Vector operations (SCAL, AXPY, DOT) are done component-by-component

For matrix-vector products, we launch a separate task for each matrix piece and reduce into a single vector



These are the only two operations needed
for **Krylov subspace methods**: CG, BiCG, MINRES, GMRES, ...

Multiple Right-Hand Sides

$$A\mathbf{x}_1 = \mathbf{b}_1$$

$$A\mathbf{x}_2 = \mathbf{b}_2$$

$$A\mathbf{x}_3 = \mathbf{b}_3$$

$$\begin{bmatrix} A & & \\ & A & \\ & & A \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

Related Systems

$$A\mathbf{x}_1 = \mathbf{b}_1$$

$$A\mathbf{x}_2 = \mathbf{b}_2$$

$$A\mathbf{x}_3 = \mathbf{b}_3$$

$$\begin{bmatrix} A & & \\ & A & \\ & & A \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

$$(A + \delta A_1) \mathbf{x}_1 = \mathbf{b}_1$$

$$(A + \delta A_2) \mathbf{x}_2 = \mathbf{b}_2$$

$$(A + \delta A_3) \mathbf{x}_3 = \mathbf{b}_3$$

$$\begin{bmatrix} A + \delta A_1 & & \\ & A + \delta A_2 & \\ & & A + \delta A_3 \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

Implementing Krylov Algorithms

```
1 template <typename ENTRY_T>
2 class CGSolver {
3
4     \implname{}:Planner<ENTRY_T> &planner;
5     static constexpr vec_id SOL = 0;
6     static constexpr vec_id RHS = 1;
7     vec_id P;
8     vec_id Q;
9     vec_id R;
10    Scalar<ENTRY_T> res; // squared residual
11
12 public:
13
14     explicit CGSolver(Planner<ENTRY_T> &planner)
15         : planner(planner) {
16         assert(planner.is_square());
17         assert(!planner.has_preconditioner());
18         P = planner.allocate_workspace_vector();
19         Q = planner.allocate_workspace_vector();
20         R = planner.allocate_workspace_vector();
21         planner.copy(P, RHS);
22         planner.copy(R, RHS);
23         res = planner.dot(R, R);
24     }
25
26     void step() {
27         planner.matmul(Q, P);
28         Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
29         planner.axpy(SOL, res / p_norm, P);
30         planner.axpy(R, -res / p_norm, Q);
31         Scalar<ENTRY_T> new_res = planner.dot(R, R);
32         planner.xpay(P, new_res, res, R);
33         res = new_res;
34     }
35
36     Scalar<ENTRY_T> get_convergence_measure() const
37     {
38         return res;
39     }
40 }; // class CGSolver<ENTRY_T>
```

Implementing Krylov Algorithms

```
1 template <typename ENTRY_T>
2 class CGSolver {
3
4     \implname{}::Planner<ENTRY_T> &planner;
5     static constexpr vec_id SOL = 0;
6     static constexpr vec_id RHS = 1;
7     vec_id P;
8     vec_id Q;
9     vec_id R;
10    Scalar<ENTRY_T> res; // squared residual
11
12 public:
13
14     explicit CGSolver(Planner<ENTRY_T> &planner)
15         : planner(planner) {
16         assert(planner.is_square());
17         assert(!planner.has_preconditioner());
18         P = planner.allocate_workspace_vector();
19         Q = planner.allocate_workspace_vector();
20         R = planner.allocate_workspace_vector();
21         planner.copy(P, RHS);
22         planner.copy(R, RHS);
23         res = planner.dot(R, R);
24     }
25
26     void step() {
27         planner.matmul(Q, P);
28         Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
29         planner.axpy(SOL, res / p_norm, P);
30         planner.axpy(R, -res / p_norm, Q);
31         Scalar<ENTRY_T> new_res = planner.dot(R, R);
32         planner.xpay(P, new_res, res, R);
33         res = new_res;
34     }
35
36     Scalar<ENTRY_T> get_convergence_measure() const
37     {
38         return res;
39     }
40 }; // class CGSolver<ENTRY_T>
```

```
void step() {
    planner.matmul(Q, P);
    Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
    planner.axpy(SOL, res / p_norm, P);
    planner.axpy(R, -res / p_norm, Q);
    Scalar<ENTRY_T> new_res = planner.dot(R, R);
    planner.xpay(P, new_res, res, R);
    res = new_res;
}
```

Implementing Krylov Algorithms

```
1 template <typename ENTRY_T>
2 class CGSolver {
3
4     \implname{}:Planner<ENTRY_T> &planner;
5     static constexpr vec_id SOL = 0;
6     static constexpr vec_id RHS = 1;
7     vec_id P;
8     vec_id Q;
9     vec_id R;
10    Scalar<ENTRY_T> res; // squared residual
11
12 public:
13
14     explicit CGSolver(Planner<ENTRY_T> &planner)
15         : planner(planner) {
16         assert(planner.is_square());
17         assert(!planner.has_preconditioner());
18         P = planner.allocate_workspace_vector();
19         Q = planner.allocate_workspace_vector();
20         R = planner.allocate_workspace_vector();
21         planner.copy(P, RHS);
22         planner.copy(R, RHS);
23         res = planner.dot(R, R);
24     }
25
26     void step() {
27         planner.matmul(Q, P);
28         Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
29         planner.axpy(SOL, res / p_norm, P);
30         planner.axpy(R, -res / p_norm, Q);
31         Scalar<ENTRY_T> new_res = planner.dot(R, R);
32         planner.xpay(P, new_res, res, R);
33         res = new_res;
34     }
35
36     Scalar<ENTRY_T> get_convergence_measure() const
37     {
38         return res;
39     }
40 }; // class CGSolver<ENTRY_T>
```

note: solver algorithms do not need to be aware of our new abstractions

```
void step() {
    planner.matmul(Q, P);
    Scalar<ENTRY_T> p_norm = planner.dot(P, Q);
    planner.axpy(SOL, res / p_norm, P);
    planner.axpy(R, -res / p_norm, Q);
    Scalar<ENTRY_T> new_res = planner.dot(R, R);
    planner.xpay(P, new_res, res, R);
    res = new_res;
}
```


Benchmarks: Performance Comparison

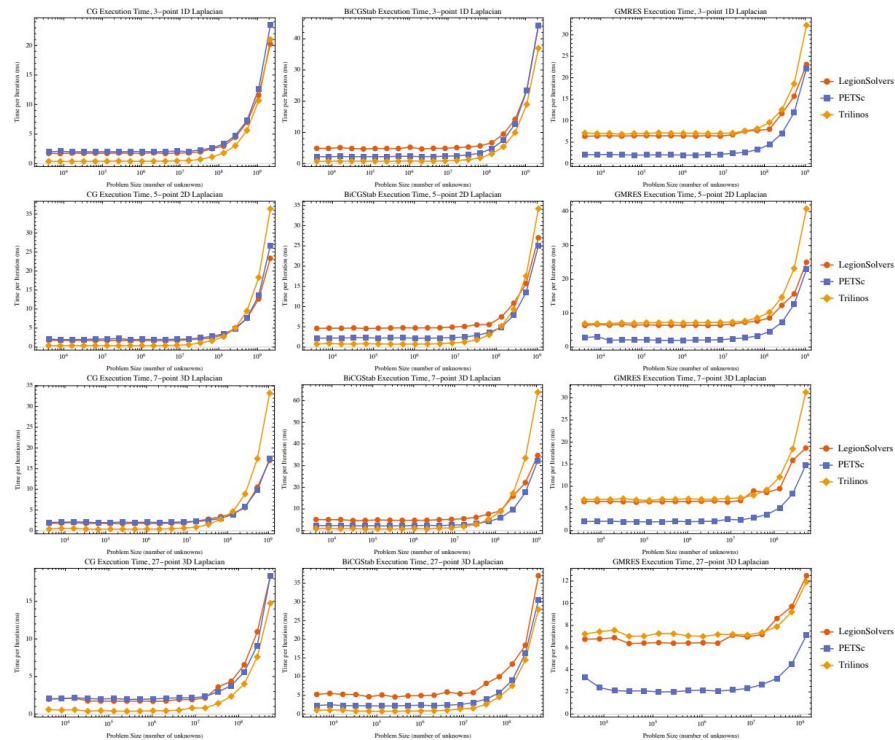
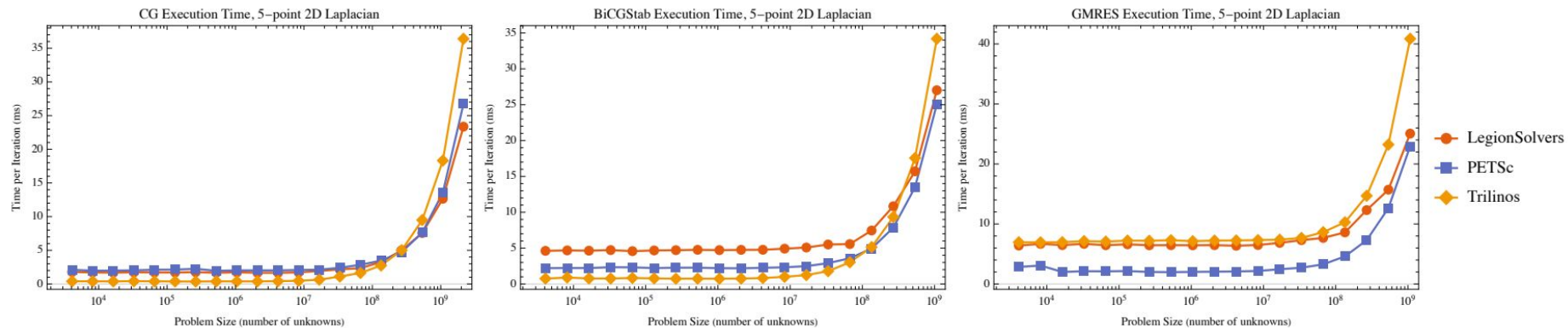


Fig. 8. Performance of KSMs (CG, BiCGStab, GMRES) applied to various stencil matrices (3-point 1D, 5-point 2D, 7-point 3D, 27-point 3D) as implemented in LegionSolvers, PETSc, and Trilinos. 16 nodes (64 GPUs)

Benchmarks: Performance Comparison



Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \end{bmatrix}$$

Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \end{bmatrix}$$

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$$

Benchmarks: Multi-Operator Systems

$$\begin{bmatrix} A \end{bmatrix} \begin{bmatrix} \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \end{bmatrix}$$

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \end{bmatrix}$$

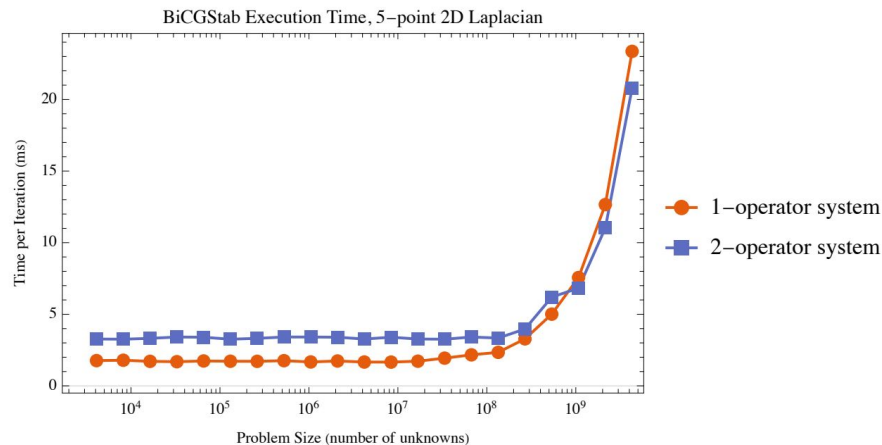


Fig. 10. Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^n \times 2^n$ grid, measured as a function of n , formulated as either a single-operator system (red) or a multi-operator system (blue).

Benchmarks: Dynamic Load-Balancing

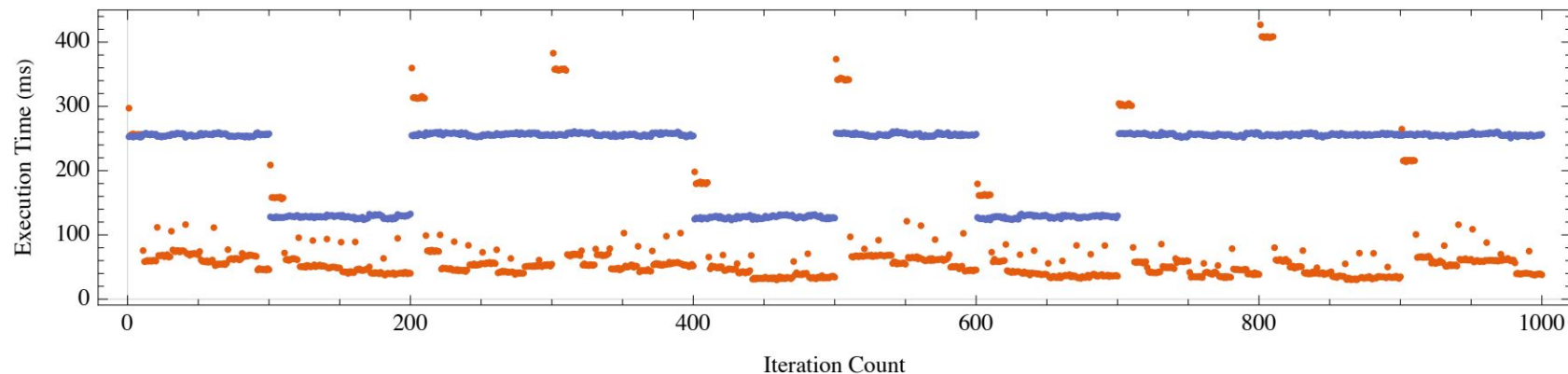


Fig. 9. Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

Benchmarks: Dynamic Load-Balancing

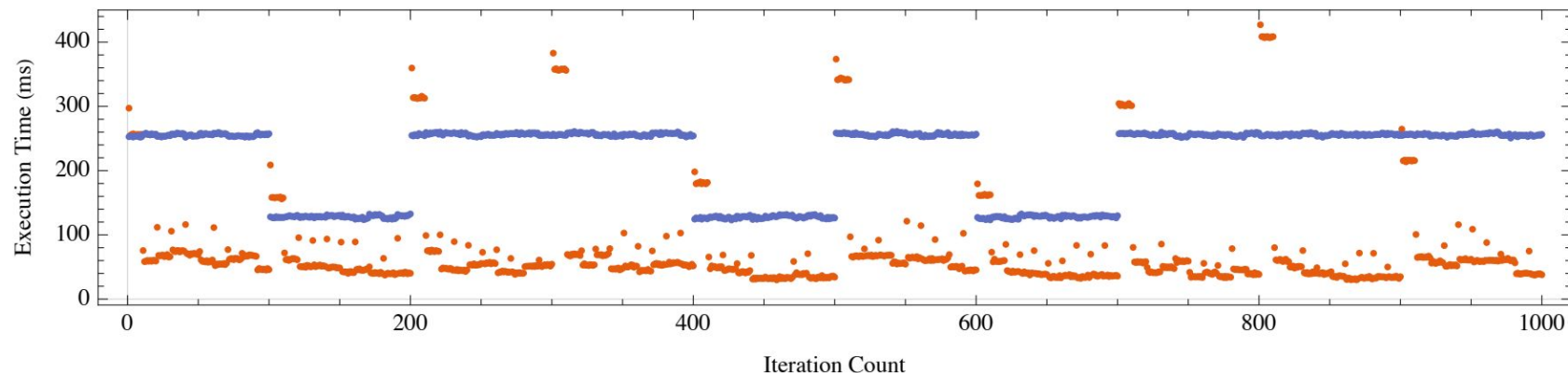


Fig. 9. Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task that occupies a uniformly random number of its cores

Benchmarks: Dynamic Load-Balancing

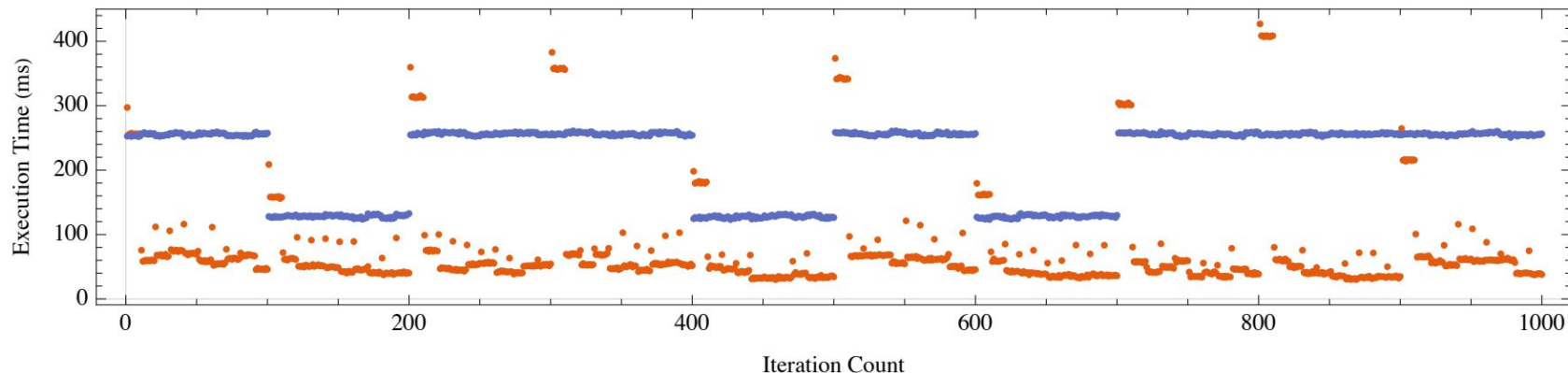


Fig. 9. Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task
that occupies a uniformly random number of its cores
Number of occupied cores is randomized every 100th CG iteration

Benchmarks: Dynamic Load-Balancing

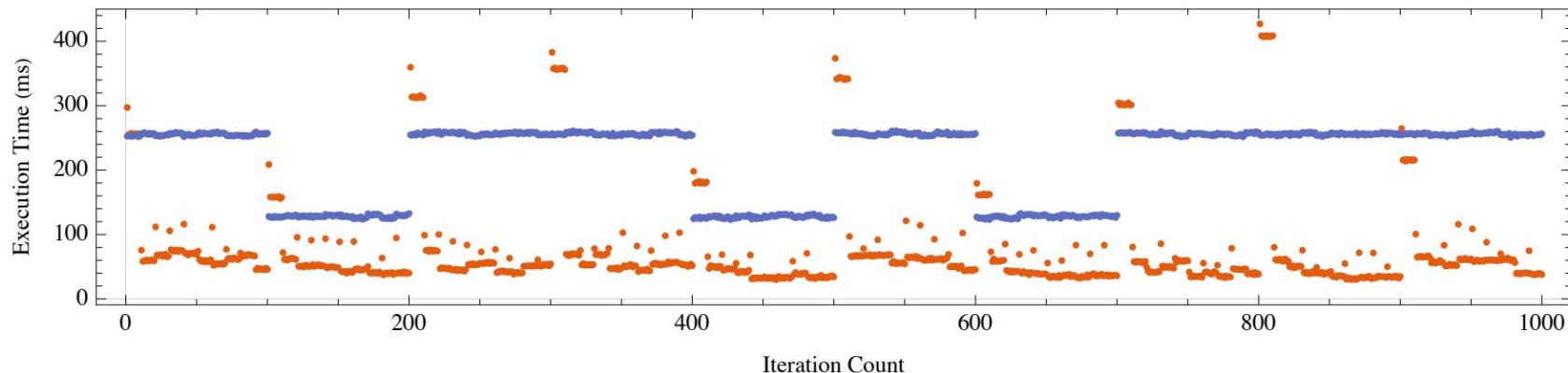


Fig. 9. Execution time per iteration of BiCGStab using a 5-point Laplacian stencil on a $2^{16} \times 2^{16}$ grid with (red) and without (blue) dynamic load-balancing.

In addition to a BiCGStab solve, each CPU also runs a dummy task that occupies a uniformly random number of its cores
Number of occupied cores is randomized every 100th CG iteration
We run a dynamic load-balancing strategy every 10th CG iteration

What's Next?

- multiple ghost region optimization: communicate twice as much, half as often
 - use dependent partitioning to compute *double* images/preimages
 - each node computes both $A\mathbf{v}$ and $A^2\mathbf{v}$ without intermediate communication

What's Next?

- multiple ghost region optimization: communicate twice as much, half as often
 - use dependent partitioning to compute *double* images/preimages
 - each node computes both $A\mathbf{v}$ and $A^2\mathbf{v}$ without intermediate communication
- symbolic solver execution: run solver algorithm ahead and identify matrix-vector operations that can be done in parallel

What's Next?

- multiple ghost region optimization: communicate twice as much, half as often
 - use dependent partitioning to compute *double* images/preimages
 - each node computes both $A\mathbf{v}$ and $A^2\mathbf{v}$ without intermediate communication
- symbolic solver execution: run solver algorithm ahead and identify matrix-vector operations that can be done in parallel
- better dynamic load-balancing strategies

What's Next?

- multiple ghost region optimization: communicate twice as much, half as often
 - use dependent partitioning to compute *double* images/preimages
 - each node computes both $A\mathbf{v}$ and $A^2\mathbf{v}$ without intermediate communication
- symbolic solver execution: run solver algorithm ahead and identify matrix-vector operations that can be done in parallel
- better dynamic load-balancing strategies
- more realistic dynamic load-balancing benchmarks